S/W Development Completion Report

Project Name			
Author		**Approver**	
Team			

Contents

▪ Revision History

Version	Date	Revised contents	Author	Approver
1.0	19.12.2025	Initial Release4 Completion Report; file persistence implementation	Magnus Jaaska	

▪ Terms and Abbreviation

DLD	Detailed Level Design
SRS	Software Requirements Specification
UI	User Interface
I/O	Input/Output (file operations)
RAII	Resource Acquisition Is Initialization
CSV	Comma-Separated Values (semicolon in our case)
STL	Standard Template Library

▪ References

- [1] Magnus Jaaska. Currency Exchange System – Software Requirements Specification. Release 1. TalTech. October 2025.
- [2] Magnus Jaaska. Currency Exchange System – Detailed Level Design. Version 3.0. TalTech. December 2025.
- [3] TalTech. Fundamentals of C++ Programming. Course Materials. Fall 2025.

1 Project Result

1.1 Development Period and Responsibilities

<Specify the Development Period and the Responsibilities in Table 1-1. Development Type should be selected out of System Project, S/W Research Project, Outsourcing Project, New Development Project and Up-grade Development Project.>

Table 1-1. Development Term and Responsibilities

Development Period	October 2025 - December 2025 (3 months)		
Development Type	New Development Project (Educational Project)		
Responsibility	**Team**	**Position**	**Name**
Development PL	Compile Commander	Student/Developer	Magnus Jaaska
S/W Development PL	Compile Commander	Student/Developer	Magnus Jaaska

1.2 Project Achievements

Table 1-2. Project Achievements

Item	Achievements
Basic Achievements	Successfully implemented 4-release incremental development methodology • Delivered fully functional currency exchange system with persistent storage • Achieved 100% requirement coverage from SRS • Zero critical bugs in final release
Acquired Technologies	C++ file I/O operations (ifstream, ofstream) • Text file parsing and validation techniques • STL containers (std::map, std::vector) • Error handling and exception strategies • Layered software architecture design • UML modeling (class, sequence, activity diagrams)
Transferable Technologies	Generic file persistence layer implementation • CSV/text file parsing framework • Repository pattern with abstraction layer • Defensive programming techniques for data validation
Documentation	Complete SRS (Release 1) • Detailed Level Design (Releases 2-4) • Mermaid UML diagrams for all major components • Comprehensive inline code documentation

1.3 Development Effect and Utilization of Results

Development Effects:

1. *Educational Impact:*
    - *Demonstrated practical application of C++ programming concepts*
    - *Applied software engineering principles (SOLID, separation of concerns)*
    - *Gained experience with incremental development and version control*
    - *Practiced technical documentation writing*
2. *Technical Benefits:*
    - *Reusable architecture for similar data management applications*
    - *Robust error handling framework applicable to other file I/O projects*
    - *Clean separation of concerns enables easy maintenance and extension*
3. *Performance:*
    - *Fast in-memory operations during runtime*
    - *Efficient file I/O with complete save/load in milliseconds for typical datasets*
    - *Graceful degradation with malformed data (skip invalid lines)*
4. *Utilization Fields:*
    - *Financial Applications: Template for currency exchange, stock trading, or banking systems*
    - *Data Management Systems: Generic pattern for loading/saving structured data*
    - *Educational Projects: Reference implementation for C++ course assignments*
    - *Small Business Tools: Foundation for point-of-sale or accounting software*

2 Development History

2.1 Development S/W Structure

2.1 Development S/W Structure

Directory Structure:

```
project-2025-compilecommander/
├── bin/                          # Compiled executables
│   └── exchange_demo             # Main application binary
├── build/                        # Object files (.o)
├── include/                      # Header files (.h)
│   ├── console_ui.h
│   ├── currency_repository.h     # [MODIFIED R4] Added persistence
methods
│   ├── exchange_manager.h
│   ├── iclock.h
│   ├── receipt.h
│   ├── report.h
│   └── transaction.h
├── src/                          # Implementation files (.cpp)
│   ├── console_ui.cpp
│   ├── currency_repository.cpp  # [MODIFIED R4] Implemented file I/O
│   ├── exchange_manager.cpp
│   ├── iclock.cpp
│   ├── main.cpp                  # [MODIFIED R4] Added lifecycle
management
│   ├── receipt.cpp
│   ├── report.cpp
│   └── transaction.cpp
├── docs/                         # Documentation
│   ├── release-1/                # SRS
│   ├── release-2/                # Initial DLD
│   └── release-3/                # Validation & exceptions
├── release4/                     # Release 4 documentation
│   ├── DLD_Release4.md           # [NEW] Detailed Level Design
│   └── SW_Development_Completion_Report_Release4.md  # [NEW] This
document
├── data.txt                      # [NEW R4] Persistent storage file
├── Makefile                      # Build configuration
└── README.md                     # Project overview
```

2.2 Development Deliverables

Release 1 (October 2025): Foundation & Requirements

- Software Requirements Specification (SRS)
- Project structure setup
- Makefile and build system
- Initial README documentation

Release 2 (October 2025): Architecture & Implementation

- Three-layer architecture implementation
- Repository pattern with interfaces
- Core exchange logic
- Basic UI implementation
- Detailed Level Design (DLD) v1.0

Release 3 (November 2025): Validation & Error Handling

- Input validation at all layers
- Custom exception classes (InvalidInput, InvalidData, InsufficientReserve, MissingRate)
- Exception handling policy
- Activity and sequence diagrams
- DLD v2.0 with validation rules and behavioral models

Release 4 (December 2025): Persistent Storage ⭐

- File I/O implementation:
  - `loadAll(filename)` - Load balances from file
  - `saveAll(filename)` - Save balances to file
  - `parseLine()` - Parse and validate file lines
  - `toLine()` - Format balance entries
- File format specification (semicolon-separated values)
- Defensive parsing (skip invalid lines with warnings)
- Startup/shutdown lifecycle management
- **DLD v3.0** with:
  - Mermaid class diagrams
  - Activity diagrams for file operations
  - Sequence diagram for persistence lifecycle
  - File format specification
  - Architectural decision documentation
- **S/W Development Completion Report** (this document)
- Persistent data file (`data.txt`)

2.3 Development Environment

Table 2-1. Hardware Environment

Type	Operating System	Major Use	Remarks
Development PC	*macOS 15.1 (Darwin 25.1.0)*	*Development, testing, compilation*	*Apple Silicon (M-series)*

Table 2-2. Software Environment

Type	Model & Spec.	Major Use	Remarks
Compiler	*g++ (Apple clang version 16.0.0)*	*C++ compilation*	*C++20 standard*
Build System	*GNU Make 3.81*	*Automated build process*	*Makefile with targets*
Version Control	*Git 2.45+*	*Source code management*	*GitHub repository*
IDE	*Visual Studio Code*	*Code editing, debugging*	*C++ extensions installed*